

SIMATS ENGINEERING

ASSESSMENT TOOL 2

**COURSE CODE: CSA14
DESIGN**

COURSE NAME: COMPILER

COURSE OUTCOME COVERED

CO3: Analyze syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree. (BL4)

Assessment Tool Weightage: 20%

QUESTIONS:

Total Marks:100

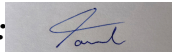
Annexure A

Debugging

Code of Conduct:

*I **Tamil Elakiya S (192524443)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:



Q. No	Understanding of Debugging Concepts (25)	Error Identification(25)	Root Causes Analysis (20)	Error Corrections and Solution Design(20)	Testing and Validation(10)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	/ 25	/ 20	/ 20	/ 10	/ 100

Annexure A

Debugging

Q1 Consider the following syntax-directed definition:

$$E \rightarrow E1 + T \quad (E.val = T.val)$$
$$E \rightarrow T \quad \{ E.val = T.val \}$$
$$T \rightarrow num \quad \{ T.val = num.lexval \}$$

For the input:

3 + 5

the compiler produces:

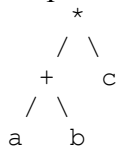
$E.val = 5$

Debug the given syntax-directed definition. Identify the incorrect semantic rule, explain its effect on attribute evaluation, provide the corrected rule, and determine the correct value of the expression.

Q2 . For the expression:

a + b * c

the compiler constructs the following syntax tree:



The compiler subsequently generates:

$t1 = a + b$

$t2 = t1 * c$

Debug the syntax-tree construction. Analyze the relationship between operator precedence and the generated tree, construct the corrected syntax tree, and generate the correct intermediate representation.

Q3. For the expression:

3 + 4 * 5

using:

$$E \rightarrow E + T \mid T$$
$$T \rightarrow T * F \mid F$$
$$F \rightarrow num$$

the compiler annotates the root with:

$E.val = 35$

Analyze the annotated parse tree. Trace the synthesized attributes from the leaves to the root, identify the incorrect computation, and determine the correct value of the expression.

Q4 . The symbol table contains:

Identifier	Type
a	int
b	float

Identifier	Type
c	int
x	float

For the statement:

```
x = a + b * c;
```

the compiler reports:

Type Error: incompatible operands

Debug the type-checking process. Analyze each subexpression, determine the type of $b * c$ and $a + (b * c)$, identify where type conversion is required, and determine whether the assignment to x is valid.

Q5. Consider:

```
int a;
float b;
int x;
```

and the statements:

```
x = a + b;
b = a + x;
```

The type checker reports both statements as:

Valid

and does not generate any conversion.

Analyze the type-checking results. Determine the type of each expression, identify the required conversions, and explain how the type checker should process each assignment.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Debugging Concepts	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Error Identification	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Root Causes Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning

Error Correction & Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Testing and Validation	10	Clear, well-structu red, and easy to understand	Organized and understanda ble presentation	Limited clarity and structure	Poor clarity; difficult to understand

Name: Tamil Elakiyas

Reg.No: 192524443

Course: Compiler Design

course

Code: CSA1408

C03

Assessment Tool - 2

Debugging Tool .

1. Error:

The semantic rule associated with the production $E \rightarrow E_1 + T$ is incorrectly defined as $\{E.val = T.val\}$

↳ Synthesized attributes are computed from the attribute values of a node's children.

↳ In the incorrect rule $\{E.val = T.val\}$, the semantic action completely ignores the value synthesized by E_1 ($E_1.val$).

↳ During evaluation of $3+5$, E_1 evaluates to 3 and T evaluates to 5. The rule discards 3 and assigns $E.val = 5$.

Corrected Semantic Rule:

$$E \rightarrow E_1 + T \quad \{E.val = E_1.val + T.val\}$$

Correct Value Computation:

1. $T \rightarrow \text{num} \Rightarrow T.val = 3$

2. $E_1 \rightarrow T \Rightarrow E_1.val = T.val = 3$

3. $T \rightarrow \text{num} \Rightarrow T.val = 5$

4. $E \rightarrow E_1 + T \Rightarrow E.val = 3 + 5 = 8$

2. Error:

The syntax tree constructed places the addition operator (+) below the multiplication operator (*).

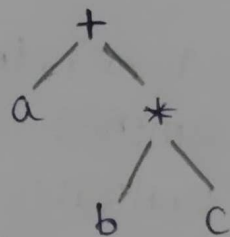
* In an Abstract Syntax Tree (AST), subtrees located at lower depth levels are evaluated first.

* Constructing + lower than * assigns higher precedence to addition.

* Consequently, the compiler evaluates $a + b$ first (generating $t_1 = a + b$) and then multiplies by c ($t_2 = t_1 * c$).

* Standard programming language grammar dictates that multiplication * has higher precedence than addition +.

Corrected Syntax Tree:



Correct Intermediate Representation:

$$t_1 = b * c;$$

$$t_2 = a + t_1;$$

3. Error:

The compiler incorrectly computes the root attribute as $E.val = 35$ for the expression $3 + 4 * 5$.

Parse Tree & Attribute Tree:

$$1. F \rightarrow \text{num}(3) \Rightarrow F.val = 3$$

$$T_1 \rightarrow F \Rightarrow T_1.val = 3$$

$$E_1 \rightarrow T_1 \Rightarrow E_1.val = 3$$

$$F \rightarrow \text{num}(4) \Rightarrow F.\text{val} = 4;$$

$$T_2 \rightarrow F \Rightarrow T_2.\text{val} = 4$$

$$3. F \rightarrow \text{num}(5) \Rightarrow F.\text{val} = 5;$$

$$4. T_3 \rightarrow T_2 * F \Rightarrow T_3.\text{val} = 4 \times 5 = 20$$

$$5. E_2 \rightarrow E_1 + T_3 \Rightarrow E_2.\text{val} = 3 + 20 = 23$$

Root Cause:

The erroneous result 35 occurred because the semantic evaluation performed a left-to-right evaluation $(3+4) \times 5 = 35$, disregarding the operator hierarchy enforced by the context-free grammar ($E \rightarrow E+T \mid T, T \rightarrow T * F \mid F$).

Correct Result:

$$E.\text{val} = 23$$

Q4. Error:

The compiler raised a false positive
`TypeError: incompatible operands` for the expression
`x = a + b * c`.

- Operand Types: $a : \text{int}$
 $b : \text{float}$
 $c : \text{int}$
 $x : \text{float}$

• Subexpression 1 ($b * c$):

Operands are float and int. Type hierarchy requires widening c from int to float via implicit coercion (`intToFloat(c)`).

Resulting type: float.

* Subexpression $2(a + (b * c))$:

Operands are a (int) and the result of $(b * c)$ (float). a must be implicitly converted to float ($\text{intToFloat}(a)$). Resulting type: float .

• Assignment ($x = \text{expression}$): Target variable x is float and expression evaluates to float . Types match without narrowing loss.

∴ The statement is type-valid. The error occurred because the type checker lacked implicit type promotion (coercion) logic.

5. Error:

The Type checker incorrectly flagged both assignments as valid without inserting mandatory implicit conversion instructions/nodes into the AST.

• Statement 1 ($x = a + b$ where $a : \text{int}$, $b : \text{float}$, $x : \text{int}$):

1. Addition ($a + b$):

Operand a (int) must be promoted to float ($\text{intToFloat}(a)$) before addition. Addition yields float .

2. Assignment ($x = \text{result}$):

Assigning a float result to an int target (x) requires explicit or implicit narrowing coercion (floatToInt) to prevent unsafe type assignment.

• Statement 2 ($b = a + x$ where $a : \text{int}$, $x : \text{int}$, $b : \text{float}$):

1. Addition ($a + x$):

Both operands are int , yielding an int result.

2. Assignment ($b = \text{result}$):

Assigning an int result to a float target (b) requires implicit widening conversion (intToFloat).

$\therefore$ The Type checker must transform expressions into an intermediate representation containing explicit conversion nodes.
